

Let `layout_stride::mapping` with zero extent(s) accept zero strides

Document #: P3959R0
Date: 2026-01-08
Project: Programming Language C++ LEWG
Reply-to: Jacob Faibussowitsch
<jfaibussowit@nvidia.com>
Mark Hoemmen
<mhoemmen@nvidia.com>
Christian Trott
<crtrott@sandia.gov>

Contents

- 1 Author
- 2 Revision history
- 3 Introduction
- 4 Design intent of `layout_stride`
 - 4.1 Supported use cases
 - 4.2 `layout_stride::mapping` is not the most general strided layout mapping
- 5 Motivation
 - 5.1 Permit conversion of empty `layout_left` or `layout_right` mapping to `layout_stride::mapping`
 - 5.2 Simplify interoperability between Python and `mdspan`
- 6 Relaxing the precondition does not violate current requirements
- 7 Implementation
- 8 Proposed wording
 - 8.1 Increment `__cpp_lib_mdspan` feature test macro
 - 8.2 Change `[mdspan.layout.stride.cons]` 4.1
 - 8.3 Change `[mdspan.layout.stride.cons]` 7.2

§ 1 Author

- Jacob Faibussowitsch (jfaibussowit@nvidia.com) (NVIDIA)
- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)

§ 2 Revision history

- Revision 0 to be submitted by 2026-01-16

§ 3 Introduction

We propose to change `layout_stride::mapping`'s constructors to permit strides to be zero if one or more extents are zero. For example, for extents (3, 5, 0, 11), this change would permit any nonnegative strides. Currently, users would need to set the stride(s) corresponding to zero extents to an arbitrary positive value, such as 1. That would make the strides (1, 3, 1, 105) in this example.

This change has two benefits. First, it would let users convert any empty `layout_left::mapping` or `layout_right::mapping` to `layout_stride::mapping`. Second, it would prevent unnecessary precondition violations when creating `mdspan` objects that view multidimensional arrays created in Python or other languages.

This change would relax preconditions of three existing `layout_stride::mapping` constructors. It would not change what code is currently well-formed or ill-formed. The only way in which it could affect backwards compatibility is by introducing `layout_stride::mapping` instances that have zero strides. This may affect use of some libraries like the [BLAS](#) and [LAPACK](#) that forbid a zero stride, even if the corresponding matrix dimension is zero. On the other hand, `layout_left` and `layout_right` mappings can already have zero strides, so generic code that operates on the elements of an `mdspan` by calling the BLAS or LAPACK must already account for this.

§ 4 Design intent of `layout_stride`

§ 4.1 Supported use cases

The design intent of `layout_stride` is to support the following use cases.

1. Any `layout_left`, `layout_right`, `layout_left_padded`, or `layout_right_padded` mapping can be converted to a `layout_stride` mapping without loss of information or possibility of failure.
2. `layout_stride::mapping` represents exactly the set of layout mappings resulting from one or more applications of `submdspan`, starting with any `mdspan` with `layout_left`, `layout_right`, `layout_left_padded`, or `layout_right_padded` mapping.

The mapping actually supports more general conversions than (1). This is because its constructor with a `const StridedLayoutMapping&` parameter accepts any layout mapping (including user-defined mappings) for which `is_always_strided()` is true. According to the [layout mapping requirements](#), this means that the result of evaluating the layout mapping on any multidimensional index in its extents is the dot product of the index and the strides. The Notes attached to the layout mapping requirements call such a layout mapping a “strided layout [mapping].”

§ 4.2 `layout_stride::mapping` is not the most general strided layout mapping

Strided layout mappings exist that `layout_stride::mapping` cannot represent.

1. `layout_stride::mapping` does *not* support “broadcasting” layout mappings.
2. `layout_stride::mapping` does *not* support negative strides.
3. `layout_stride::mapping` is always unique. (That all the strides are positive is necessary but not sufficient in order for this to hold.)

Regarding (1), a *broadcasting layout mapping* has one or more broadcasting extents. A *broadcasting extent* is greater than one, but all indices in that extent refer to the same element. For example, if an `mdspan x` with `default_accessor` has rank 3 and extent 2 (the rightmost extent) is broadcasting, then `&x[i, j, k1] == &x[i, j, k2]` for all indices `k1` and `k2` in `[0, x.extent(2))`. One way to get a broadcasting layout mapping would be to have a strided layout with stride zero in its broadcasting extent(s).

Regarding (2), a strided layout mapping may have a negative stride as long as the corresponding extent is no greater than one. (If that extent were greater than one, then some multidimensional index would exist for which the mapping should return a negative number. The layout mapping requirements forbid this.)

Regarding (3), while `mdspan` generally permits custom nonunique layouts, the `mdspan` authors did not want a commonly used layout such as `layout_stride` to have this behavior. This is because it can be difficult to understand how to write generic algorithms for nonunique layouts, especially for algorithms that need to write to the `mdspan`’s elements.

These restrictions were always part of `layout_stride`’s design. It’s something `mdspan`’s layouts inherited from `Kokkos::View`. It’s also part of the reason why the layout mapping requirements define a strided layout mapping separately from `layout_stride`. Relaxing this would break `submdspan`.

§ 5 Motivation

§ 5.1 Permit conversion of empty `layout_left` or `layout_right` mapping to `layout_stride::mapping`

Availability of so many conversions to `layout_stride::mapping` makes it natural for users to treat `layout_stride::mapping` as a “type-erased” mapping, for example when defining stable application binary interfaces. That works fine, except when the conversion’s input mapping has a zero extent. For example, creating a `layout_right` or `layout_left` mdspan with one or more extents of zero can result in an mdspan with one or more strides of zero. For example, a `layout_right` mdspan with extents (1, 0) will have strides (0, 1), and a `layout_left` mdspan with extents (0, 1) will have strides (1, 0). This is expected behavior, and it matches implementations. For example, [this Compiler Explorer link](#) builds and runs the following example with Clang 21.1.0 and libc++, using build options `-std=c++26 -stdlib=libc++ -Wall`. [This Compiler Explorer link](#) builds and runs the same example (with just a namespace change) with the [reference mdspan implementation](#).

```
#include <cassert>
#include <mdspan>
#include <print>

template<class Layout>
using mdspan_2d = std::mdspan<float, std::dims<2>, Layout>;

int main() {
    mdspan_2d<std::layout_right> mr(nullptr, 1, 0);
    std::print("{} {}\n", mr.stride(0), mr.stride(1));
    assert(mr.stride(0) == 0);
    assert(mr.stride(1) == 1);

    mdspan_2d<std::layout_left> ml(nullptr, 0, 1);
    std::print("{} {}\n", ml.stride(0), ml.stride(1));
    assert(ml.stride(0) == 1);
    assert(ml.stride(1) == 0);

    return 0;
}
```

Conversion from these `layout_right` or `layout_left` mdspan to `layout_stride` mdspan violates the preconditions of `layout_stride::mapping`’s converting constructor, specifically `[mdspan.layout.stride.cons]` 7.2, that requires all strides to be positive.

We propose to relax this precondition. The conversion is otherwise well-formed. Neither the reference implementation nor libc++ enforces the precondition, and the conversion works as expected. [This Compiler Explorer link](#) demonstrates that with both implementations.

```
#define USE_REFERENCE_MDSPAN 1

#if defined(USE_REFERENCE_MDSPAN)
# include <https://raw.githubusercontent.com/kokkos/mdspan/single-
    header/mdspan.hpp>
```

```

#else
#  define _LIBCPP_DEBUG 1
#  include <mdspan>
#endif

#include <cassert>
#include <print>

#if defined(USE_REFERENCE_MDSPAN)
namespace md = std::experimental;
#else
namespace md = std;
#endif

template<class Layout>
using mdspan_2d = md::mdspan<float, md::dims<2>, Layout>;

int main(int, char* argv[]) {
    mdspan_2d<md::layout_right> mr(nullptr, 1, 0);
    mdspan_2d<md::layout_stride> mrs(mr);
    assert(mrs.stride(0) == 0u);
    assert(mrs.stride(1) == 1u);

    mdspan_2d<md::layout_left> ml(nullptr, 0, 1);
    mdspan_2d<md::layout_stride> mls(ml);
    assert(mls.stride(0) == 1u);
    assert(mls.stride(1) == 0u);
    return 0;
}

```

§ 5.2 Simplify interoperability between Python and `mdspan`

The past two decades have seen ever-increasing use of Python for data science, scientific computations, machine learning, and other domains that involve computations on multidimensional arrays. Many Python libraries for these domains have an implementation strategy of calling existing Fortran, C, or C++ libraries (such as the BLAS) with multidimensional arrays that are created and managed by Python code. These reasons have motivated Python developers to define common binary interfaces for multidimensional array data. Examples include

- the [Buffer Protocol](#),
- the NumPy library’s [ndarray protocol](#),
- [DLPack format](#), and the
- [CUDA Array Interface](#).

The growth of Python-based programming models and library ecosystems for domains like machine learning has also led to wide interoperability between multidimensional array formats. For example, JAX, PyTorch, TensorFlow, and XLA “Tensors” all can be

converted to and from NumPy ndarray arrays. Many such libraries also support the DLPack format. NVIDIA's [cuTile Python](#) has native support for objects that implement either the DLPack format or the CUDA Array Interface (e.g., [CuPy](#) arrays).

All these multidimensional array formats claim to provide what they call “strided” indexing. However, all of them support much more general layouts than what `layout_stride` supports, for three reasons.

1. All these formats but DLPack use byte strides, while `layout_stride` uses element strides. (One can use `layout_stride` to represent possibly nonaligned byte strides, but only in combination with a custom accessor. Please see [this pull request](#) for an example and discussion.)
2. All four formats permit zero or even negative strides, as well as positive strides that explicitly construct a nonunique layout. In contrast, `layout_stride::mapping` is always unique (`is_always_unique()` and `is_unique()` are both always true).
3. The four formats generally impose no requirements on strides for arrays with zero elements (where the product of the extents is zero). However, `layout_stride::mapping` currently does not permit zero strides, even if the corresponding extents are zero.

Adoption of this proposal would fix Reason (3) by permitting zero strides when the product of the extents is zero. Reasons (1) and (2) are out of scope, because relaxing those requirements on the strides would break the design intent of `layout_stride`. Therefore, if users want a layout mapping that can represent everything that (say) DLPack can represent, then they will need to write a custom layout mapping.

That being said, Python multidimensional array formats have a common convention to permit zero strides for zero extents. The ndarray format permits arbitrary values for strides [under two conditions](#).

1. If an extent (what Python calls a “shape”) is zero, then the corresponding stride can be arbitrary.
2. If an array has size zero, then the strides are never used, and thus all the strides can be arbitrary.

The constructor of `layout_stride::mapping` has a precondition that all strides are positive, even for zero extents. Users of the Fortran or C BLAS already are used to this convention, because the BLAS requires nonzero strides (though it supports negative strides in some cases!). However, this may be less intuitive for a Python developer.

We show below how to use the `pybind11` library to get a `layout_stride::mapping` corresponding to a given Python ndarray whose rank is known at compile time. We omit checks for two cases.

1. One extent may be -1, in which case the actual extent is to be inferred from the size and the remaining extents.
2. A nonunique input layout with positive strides, which ndarray permits but layout_stride does not.

```
template<std::size_t Rank>
std::layout_stride::mapping<std::dims<Rank>>
python_ndarray_to_cpp_mapping(
    const py::dict& array_interface,
    std::size_t bytes_per_element)
{
    auto py_strides = array_interface["strides"];
    auto py_shape = array_interface["shape"];
    using stride_type = std::intptr_t; // numpy.intp, a signed type

    bool any_extent_is_zero = false;
    for (std::size_t i = 0; i < Rank; ++i) {
        assert(py_shape[i] >= 0);
        if (py_shape[i] == 0) {
            any_extent_is_zero = true;
        } else if (py_strides[i] == 0) {
            throw unsupported_layout("Nonzero extent with zero stride")
        }
        if (py_strides[i] < 0) {
            throw unsupported_layout("One or more negative strides");
        }
    }

    auto cpp_strides =
        [&] <std::size_t... Inds> (std::index_sequence<Inds...>) {
            return std::array<std::size_t, Rank>{
                (any_extent_is_zero ?
                 size_t(1) :
                 py_strides[Inds] / bytes_per_element)...
            };
        } (std::make_index_sequence<Rank>());

    auto cpp_extents =
        [&] <std::size_t... Inds> (std::index_sequence<Inds...>) {
            return std::dims<Rank>{py_shape[Inds]...};
        } (std::make_index_sequence<Rank>());

    return std::layout_stride::mapping<std::dims<Rank>>{
        cpp_extents, cpp_strides};
}
```

Relaxing the requirement that strides be positive even if any extents are zero would simplify the code in two places (look for the “SIMPLER” comments) as follows.

```
template<std::size_t Rank>
std::layout_stride::mapping<std::dims<Rank>>
python_ndarray_to_cpp_mapping(
    const py::dict& array_interface,
```

```

std::size_t bytes_per_element)
{
    auto py_strides = array_interface["strides"];
    auto py_shape   = array_interface["shape"];
    using stride_type = std::intptr_t; // numpy.intp

    for (std::size_t i = 0; i < Rank; ++i) {
        if (py_shape[i] != 0 && py_strides[i] == 0) { // SIMPLER
            throw unsupported_layout("Nonzero extent with zero stride")
        }
        if (py_strides[i] < 0) {
            throw unsupported_layout("One or more negative strides");
        }
    }

    auto cpp_strides =
        [&] <std::size_t... Inds> (std::index_sequence<Inds...>) {
            return std::array<std::size_t, Rank>{
                py_strides[Inds] / bytes_per_element)... // SIMPLER
        };
    } (std::make_index_sequence<Rank>());

    auto cpp_extents =
        [&] <std::size_t... Inds> (std::index_sequence<Inds...>) {
            return std::dims<Rank>{py_shape[Inds]...};
        } (std::make_index_sequence<Rank>());

    return std::layout_stride::mapping<std::dims<Rank>>{
        cpp_extents, cpp_strides};
}

```

§ 6 Relaxing the precondition does not violate current requirements

In this section, we prove that relaxing the precondition to allow a stride of 0 for empty extents does not violate `layout_stride::mapping`'s requirements, and therefore does not necessitate a new mapping type.

A `layout_stride::mapping` currently satisfies the following properties.

1. It is always unique. That is,
 - a. `is_always_unique()` is true, and
 - b. `is_unique()` is true (for any mapping with a valid extents object).
2. If it has rank zero or if `extents_type::static_extent(r)` is zero for any rank index r of `extents()`, then it is always exhaustive. That is,
 - a. `is_always_exhaustive()` is true, and

b. `is_exhaustive()` is true (for any mapping with a valid extents object).

Uniqueness means that every multidimensional index in the mapping's extents must map to a distinct offset in `[0, required_span_size())`. That is, the mapping must be *injective*.

Exhaustiveness means that for each offset in `[0, required_span_size())`, there must exist a multidimensional index in the mapping's extents that maps to that offset. That is, the mapping must be *surjective*.

Relaxing the constraints on strides to allow a value of zero if and only if there is at least one extent of zero preserves both uniqueness and exhaustiveness. In the case where an extent is zero, the multidimensional index set is empty, and the mapping becomes a function from the empty set to the empty set. That makes the mapping both injective and surjective vacuously.

§ 7 Implementation

As shown above, both the reference `mdspan` implementation and `libc++`'s implementation actually implement this proposal by not checking the preconditions and by producing a valid `layout_stride::mapping`. In a hypothetical implementation that does check the preconditions, the only required changes would be removing these checks from three `layout_stride::mapping` constructors.

§ 8 Proposed wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.

§ 8.1 Increment `__cpp_lib_mdspan` feature test macro

In `[version.syn]`, increase the value of the `__cpp_lib_mdspan` macro by replacing `YYMMML` below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_mdspan YYYYMMML // also in <mdspan>
```

§ 8.2 Change `[mdspan.layout.stride.cons]` 4.1

Change `[mdspan.layout.stride.cons]` as follows.

```
template<class OtherIndexType>
constexpr mapping(const extents_type& e, span<OtherIndexType, _rank__>
    s) noexcept;
template<class OtherIndexType>
constexpr mapping(const extents_type& e, const array<OtherIndexType,
    _rank__>& s) noexcept;
```

3 Constraints:

- (3.1) — `is_convertible_v<const OtherIndexType&, index_type>` is true, and
- (3.2) — `is_nothrow_constructible_v<index_type, const OtherIndexType&>` is true.

4 Preconditions:

- (4.1) — ~~The result of converting $s[i]$ to `index_type` is greater than 0 for all i in the range $[0, rank_)$.~~ Let σ_i be the result of converting $s[i]$ to `index_type`. Then, for all i in the range $[0, rank_)$, if the multidimensional index space e is empty, σ_i is greater than or equal to zero, otherwise σ_i is greater than zero.
- (4.2) — `REQUIRED-SPAN-SIZE( $e$ ,  $s$ )` is representable as a value of type `index_type` ([basic.fundamental]).
- (4.3) — If `rank_` is greater than 0, then there exists a permutation P of the integers in the range $[0, rank_)$, such that `$s[p_i] \geq s[p_{i-1}] * e.extent(p_{i-1})$` is true for all i in the range $[1, rank_)$, where p_i is the i^{th} element of P .

[Editor's note: This definition permits strides to be zero if their corresponding extents are zero, because the permutation can be selected to move the zero strides to the front of the list of strides. For example, suppose that the extents are (2, 3, 0, 7, 0, 13) and the strides are (1, 2, 0, 30, 0, 2310). If the permutation is (2, 3, 0, 4, 1, 5), then the permuted extents are (0, 0, 2, 3, 7, 13) and the permuted strides are (0, 0, 1, 2, 30, 2310).]

[Note 1: For `layout_stride`, this condition is necessary and sufficient for `is_unique()` to be true. — end note]

§ 8.3 Change [mdspan.layout.stride.cons] 7.2

Change [mdspan.layout.stride.cons] as follows, to permit conversion from `layout_{left,right}::mapping` with some zero extents (or any strided layout) to `layout_stride::mapping`.

```
template<class StridedLayoutMapping>
constexpr explicit(see below)
mapping(const StridedLayoutMapping& other) noexcept;
```

6 Constraints:

- (6.1) — `layout-mapping-alike<StridedLayoutMapping>` is satisfied.
- (6.2) — `is_constructible_v<extents_type, typename StridedLayoutMapping::extents_type>` is true.
- (6.3) — `StridedLayoutMapping::is_always_unique()` is true.
- (6.4) — `StridedLayoutMapping::is_always_strided()` is true.

7 Constraints:

- (7.1) — StridedLayoutMapping meets the layout mapping requirements ([mdspan.layout.reqmts]);;
- (7.2) — ~~other.stride(*r*) > 0 is true for every rank index *r* of extents();~~ for every rank index *r* of extents(), if the multidimensional index space other.extents() is empty, other.stride(*r*) is greater than or equal to zero, otherwise other.stride(*r*) is greater than zero;
- (7.3) — other.required_span_size() is representable as a value of type index_type ([basic.fundamental]);; and
- (7.4) — `OFFSET(other) == 0` is true.